

Spread Construction in Pairs Trading

spread_builder.py — Mathematical Notes & Documentation

Cointegration-Based Spread, OLS Hedge Ratio, and Kalman Filter Interface

Project Dark-Wing | July 26, 2026

Abstract. Once two or more assets are confirmed cointegrated, their *spread* is the stationary linear combination that oscillates around a fixed mean. This document derives three spread constructions used in the `SpreadBuilder` class: OLS-estimated spread (Engle-Granger style), fixed-beta spread (from Johansen or Kalman), and the Johansen ECT spread. The key output, a stationary S_t series, feeds directly into the `HalfLife`, `ZScore`, and `Kalman_Filter` modules.

Why Do We Build a Spread?

After the Johansen test (Step 3) confirms cointegration rank $r^* \geq 1$, we know a stationary linear combination $S_t = \beta' \mathbf{X}_t$ exists. This spread is the *mean-reverting signal* we trade.

Key Intuition

If two stocks move together in the long run, their price difference will deviate from equilibrium occasionally but always return. This deviation *is* the spread S_t , and trading it is *statistically arbitrage*.

Three modes are supported:

- **OLS:** Estimate β from data (2-asset).
- **Fixed-beta:** Supply β from Johansen/Kalman.
- **Johansen ECT:** Multi-asset matrix spread.

Mode 1: OLS Spread (2-Asset)

The Cointegrating Regression

For two log-price series Y_t and X_t , the cointegrating regression is:

Formula 1 — Cointegrating Regression

$$Y_t = \alpha + \beta X_t + \varepsilon_t \quad (1)$$

If Y_t and X_t are cointegrated, then $\varepsilon_t = S_t$ is $I(0)$ (stationary). β is the *hedge ratio*.

OLS Estimation of Hedge Ratio

We minimise the sum of squared residuals:

Formula 2 — OLS in Matrix Form

$$\hat{\theta} = \begin{bmatrix} \hat{\alpha} \\ \hat{\beta} \end{bmatrix} = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y} \quad (2)$$

where $\mathbf{X} = [\mathbf{1}, X_t]$ (design matrix), $\mathbf{y} = Y_t$ (response).

Spread Construction

Once $\hat{\alpha}$ and $\hat{\beta}$ are estimated:

Formula 3 — OLS Spread

$$S_t = Y_t - \hat{\alpha} - \hat{\beta} X_t \quad (3)$$

S_t should be $I(0)$: verified by ADF/KPSS in `VECM` diagnostics.

Financial meaning. $\hat{\beta}$ tells us how many units of X to hold short for each unit of Y held long. This is the *dollar-neutral* hedge ratio. A $\beta > 1$ means X is the more expensive leg; we hold less of it per unit of Y .

Mode 2: Fixed-Beta Spread

When β is supplied externally (e.g. Johansen's β or a Kalman-filtered $\hat{\beta}_t$), we skip OLS:

Formula 4 — Fixed-Beta Spread

$$S_t = Y_t - \beta X_t, \quad \alpha = 0 \quad (4)$$

No intercept is removed. This is appropriate when the cointegrating vector already encodes the long-run mean.

When to use. Use `mode='fixed_beta'` when you have a Johansen-estimated β from `johansen.py`, or when the Kalman filter is actively updating β_t at each timestep.

Mode 3: Johansen ECT (Multi-Asset)

The ECT from Johansen

For m assets with cointegrating rank r^* , Johansen provides the $(m \times r^*)$ cointegrating matrix β . Each column is a cointegrating vector.

Formula 5 — Johansen ECT Spread

$$\mathbf{S}_t = \beta' \mathbf{X}_t \quad \text{shape: } (r^* \times 1) \quad (5)$$

Each row $S_{j,t} = \beta'_j \mathbf{X}_t$ is a separate cointegrating spread (ECT). All r^* ECT columns are stationary.

In matrix form for all T observations:

$$\underbrace{\mathbf{S}}_{T \times r^*} = \underbrace{\mathbf{X}}_{T \times m} \underbrace{\beta}_{m \times r^*}$$

Financial meaning. Each ECT column is a portfolio of all m assets with weights given by the corresponding column of β . Portfolio j is orthogonal to the unit-root component of $\mathbf{X}_t$, so it is stationary.

Hedge Ratio Summary Table

Mode	β source	Intercept?
OLS	Estimated from data	Yes ($\hat{\alpha}$)
fixed_beta	Supplied (Johansen/Kalman)	No
johansen_ect	Johansen β matrix	No

Kalman Filter Interface Contract

The `Kalman_Filter/` module requires three aligned series:

Interface: SpreadBuilder → Kalman

- `spread_series()` → S_t — observation for filter.
- `y_series()` → Y_t — dependent leg.
- `x_series()` → X_t — independent leg.

All three share the same `DatetimeIndex`. In `complete_kalman_filter.py`: `correction(y=Y_t, x=X_t)` produces the Kalman spread $s_t = Y_t - \hat{\beta}_t \cdot X_t$.

Code Flowchart — SpreadBuilder

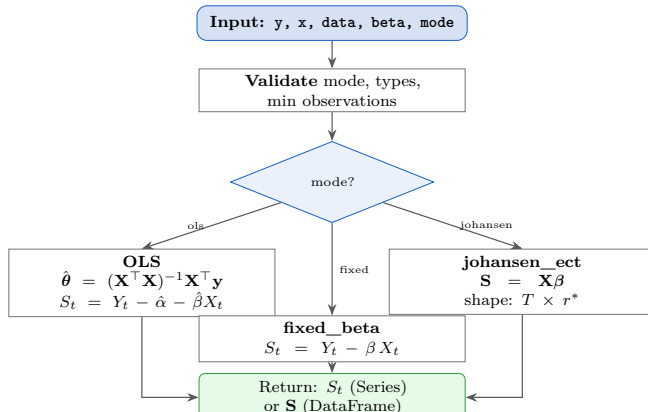


Figure 1: Execution flowchart for `SpreadBuilder.build()`.

Worked Example

Setup

Log-prices: $Y = [2.30, 2.64, 2.08, 2.77, 2.49]$, $X = [3.00, 3.40, 2.90, 3.50, 3.20]$.
Mode: **OLS**, include intercept.

Step 1 — Design matrix:

$$\mathbf{X} = \begin{bmatrix} 1 & 3.00 \\ 1 & 3.40 \\ 1 & 2.90 \\ 1 & 3.50 \\ 1 & 3.20 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 2.30 \\ 2.64 \\ 2.08 \\ 2.77 \\ 2.49 \end{bmatrix}$$

Step 2 — OLS estimate:

$$\hat{\theta} \approx \begin{bmatrix} -0.12 \\ 0.79 \end{bmatrix} \Rightarrow \hat{\alpha} = -0.12, \hat{\beta} = 0.79$$

Step 3 — Spread:

$$S_1 = 2.30 - (-0.12) - 0.79 \times 3.00 = +0.05$$

$$S_2 = 2.64 - (-0.12) - 0.79 \times 3.40 = +0.07$$

⋮

Interpretation

$\hat{\beta} = 0.79$: for every 1 unit of Y long, we short 0.79 units of X . The spread $S_t \approx 0$ with small deviations — confirming cointegration. The intercept $\hat{\alpha} = -0.12$ absorbs the long-run level difference.

Python Implementation Snippets

OLS beta estimation

```
def _ols_beta(self) -> tuple:
    y = self._y.values
    x = self._x.values
    if self.include_intercept:
        X = np.column_stack([np.ones(len(x)), x])
        coef = np.linalg.lstsq(X, y, rcond=None)[0]
        alpha, beta = float(coef[0]), float(coef[1])
    else:
        coef = np.linalg.lstsq(
            x.reshape(-1,1), y, rcond=None)[0]
        alpha, beta = 0.0, float(coef[0])
    return alpha, beta
```

Johansen ECT construction

```
# mode = 'johansen_ect'
ect_vals = self._data.values @ self._beta_input
# shape: (T, r*)
self._ect_df = pd.DataFrame(
    ect_vals, index=self._data.index,
    columns=[f"ECT_{j+1}" for j in range(r)])
# primary spread = first ECT column
self._spread = self._ect_df["ECT_1"].rename("spread")
```

Pipeline Position

Step	Module	Output
1	<code>TS_stationarity_test</code>	I(1) confirmation
2	<code>VAR_lag_select</code>	k^* lags
3	<code>johansen.py</code>	r^*, β
4	<code>fit_vecm.py</code>	α, β , ECT
4.5	<code>diagnostics.py</code>	Health score gate
5a	<code>spread_builder.py</code>	S_t series
5b	<code>half_life.py</code>	τ, κ
5c	<code>zscore.py</code>	Z_t , signals
6	<code>Kalman_Filter/</code>	Dynamic $\hat{\beta}_t$